H05 - Practical Functions

ASSIGNMENT 1 | SELF-STUDY BUCKET

Guided Self-Study Tutorial

In the previous in-class lesson, we started working with functions as named, reusable pieces of code. In this self-study tutorial, we'll keep practicing functions by building small calculation tools. The main shift is that our functions will not just print information. They will **return** useful values that other code can use.

By the end, `practice.js` will include functions for calculating angles, estimating loan payments, and converting seconds into hours, minutes, and seconds.

Assignment 1 - Self-Study Bucket

This is a self-study bucket that is **for marks**. Most students should plan for about **1-2 hours** to work through the tutorial carefully, test the functions, add comments, and commit the finished work.

To earn the marks, you must complete the tutorial in `practice.js`, record roughly how much time you spent in `Notes.md`, and commit the code in your student workbook. Commits in git are a two-step process: first, stage what changes you want to commit. Then do the actual commit.

Commit Example

```
git add .
git commit -m "Complete homework 05 practice"
```

The automated tests for this bucket check the functions exported from `practice.js`. They do not replace careful reading, but they can help confirm that your functions return the expected values.

Before You Start

1. Open the `~/homework/05/` folder in VS Code.
2. Open `practice.js` and `Notes.md`.
3. In `Notes.md`, write down when you are starting this assignment. Before you finish, you will add roughly how much time the assignment took.
4. Open the VS Code terminal and make sure the terminal is in the `~/homework/05/` folder.

5. Run the starter file with Node's "watch" mode.

Ensure you are in the ~/homework/05/ folder

```
node --watch practice.js
```

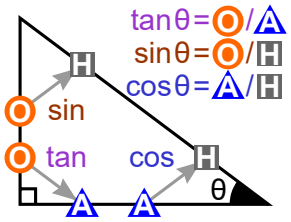
You should see the assignment heading print in the terminal.

> **Important**
>
> The bottom of `practice.js` has a comment that says `DO NOT MODIFY CONTENT BELOW THIS LINE`. Leave that section alone. The functions above that line are the only functions we need to complete.

Returning an Object from a Function

Each of the angle functions will return three related pieces of information:

1. the angle in radians
2. the angle in degrees
3. the angle in gradians



A single function can return one value. When we need one returned value to hold several smaller values, we can return an **object**.

Let's begin with a small placeholder object in the `soh()` function.

1. Find the `soh()` function in `practice.js`.

~/homework/05/practice.js

```
function soh(opposite, hypotenuse) {
  // TODO: Return the angle as an object with properties of
  //       radians, degrees, and gradians
}
```

2 Replace the TODO comment with a returned object.

~/homework/05/practice.js

```
function soh(opposite, hypotenuse) {
-  // TODO: Return the angle as an object with properties of
-  //       radians, degrees, and gradians
+  return {
+    radians: 0,
+    degrees: 0,
+    gradians: 0
+  };
}
```

This is not the correct math yet, but it gives us the correct shape. The returned object has the three properties that the rest of our code expects.

3 Add a quick test call after the `toa()` function so we can see what `soh()` returns.

~/homework/05/practice.js

```
function toa(opposite, adjacent) {
  // TODO: Return the angle as an object with properties of
  //       radians, degrees, and gradians
}

+console.log(soh(3, 5));


// Calculating Loans
```

You should see an object in the terminal output.

Output

```
{ radians: 0, degrees: 0, gradians: 0 }
```

Checkpoint

Your script should still run. At this point, `soh(3, 5)` should return an object with `radians`, `degrees`, and `gradians` properties, even though the values are still placeholders.

Calculating SOH

The first angle function is named `soh`, which comes from the trigonometry memory aid **SOH-CAH-TOA**.

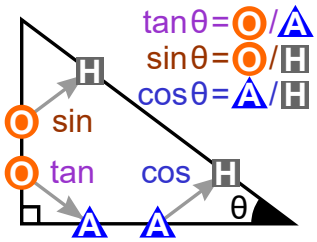


Image from Wikipedia.

For **SOH**, we use the sine relationship:

SOH

```
sine = opposite / hypotenuse
```

When we already know the opposite side and the hypotenuse, JavaScript's `Math.asin()` function can calculate the angle in **radians**.

> **Trigonometry Refresher**
>
> The regular `Math.sin()` function starts with an angle and tells us the sine ratio. In this function, we are working in the opposite direction: we already know the ratio `opposite / hypotenuse`, and we want JavaScript to tell us the angle.
>
> That is why we use `Math.asin()`. The `a` stands for **arc sine**, which is another name for inverse sine. Inverse sine answers the question: "What angle has this sine ratio?"

1. Update `soh()` so it calculates the radians value.

~/homework/05/practice.js

```
function soh(opposite, hypotenuse) {
+  const radians = Math.asin(opposite / hypotenuse);
   return {
-    radians: 0,
+    radians: radians,
     degrees: 0,
     gradians: 0
   };
}
```

2. Save the file and look at the terminal output from `console.log(soh(3, 5))`.

> You should see the `radians` property change to a number close to `0.6435011087932844`.

So far, our function knows how to calculate radians. Next, we'll compose that work with helper functions so the same radians value can be converted into degrees and gradians.

Helper Functions for Angle Conversion

A helper function is a small function that supports another function. In this case, all three angle functions will need the same two conversions:

1. radians to degrees
2. radians to gradians

Instead of copying the conversion math into `soh()`, `cah()`, and `toa()`, we'll create helper functions once and call them wherever they are needed.

1. Add two helper functions after `toa()` and before the temporary `console.log()`.

~/homework/05/practice.js

```
function toa(opposite, adjacent) {
  // TODO: Return the angle as an object with properties of
  //       radians, degrees, and gradians
}

+const toDegrees = (radians) => radians * (180 / Math.PI);
+const toGradians = (radians) => radians * (200 / Math.PI);
+
console.log(soh(3, 5));
```

These are arrow functions. They receive `radians`, calculate a converted value, and return the result automatically.

2 Use the helper functions inside `soh()`.

~/homework/05/practice.js

```
function soh(opposite, hypotenuse) {
  const radians = Math.asin(opposite / hypotenuse);
  return {
    radians: radians,
-   degrees: 0,
-   gradians: 0
+   degrees: toDegrees(radians),
+   gradians: toGradians(radians)
  };
}
```

3 Save the file and check the terminal output again.

You should see values close to this:

Output

```
{
  radians: 0.6435011087932844,
  degrees: 36.86989764584402,
  gradians: 40.96655293982669
}
```

Checkpoint

At this point, `soh(3, 5)` should return the angle in all three units. Notice that the `soh()` function does the sine-specific work, then delegates the repeated conversion work to `toDegrees()` and `toGradians()`.

Calculating CAH and TOA

Now we can complete the other two angle functions using the same pattern.

For **CAH**, we use the cosine relationship:

CAH

```
cosine = adjacent / hypotenuse
```

For **TOA**, we use the tangent relationship:

TOA

```
tangent = opposite / adjacent
```

JavaScript gives us matching inverse trigonometry functions:

1. `Math.acos()` for cosine
2. `Math.atan()` for tangent

1. Complete the `cah()` function.

~/homework/05/practice.js

```
function cah(adjacent, hypotenuse) {
-  // TODO: Return the angle as an object with properties of
-  //       radians, degrees, and gradians
+  const radians = Math.acos(adjacent / hypotenuse);
+  return {
+    radians: radians,
+    degrees: toDegrees(radians),
+    gradians: toGradians(radians)
+  };
}
```

2. Complete the `toa()` function.

~/homework/05/practice.js

```
function toa(opposite, adjacent) {
-  // TODO: Return the angle as an object with properties of
-  //       radians, degrees, and gradians
+  const radians = Math.atan(opposite / adjacent);
+  return {
+    radians: radians,
+    degrees: toDegrees(radians),
+    gradians: toGradians(radians)
+  };
}
```

3 Update the temporary test calls so we can inspect all three angle functions.

~/homework/05/practice.js

```
-console.log(soh(3, 5));
+console.log(soh(3, 5));
+console.log(cah(4, 5));
+console.log(toa(3, 4));
```

These three calls should all produce angles of about `36.87` degrees.

Checkpoint

Your `soh(3, 5)`, `cah(4, 5)`, and `toa(3, 4)` calls should now return objects with similar angle values. The exact decimals may look long, and that is fine. The tests expect JavaScript's normal floating-point results.

> We will remove the temporary `console.log()` calls before the end of the tutorial. For now, they help us see what our functions return.

Calculating a Loan Payment

The next function, `calculateLoan()`, returns an object that describes a loan:

1. the monthly payment
2. the total number of payments
3. the total amount paid

The function receives three inputs:

1. `principal`: the amount borrowed
2. `annualRate`: the yearly interest rate as a percentage
3. `years`: the length of the loan

1 Find the `calculateLoan()` function.

~/homework/05/practice.js

```
function calculateLoan(principal, annualRate, years) {
  // TODO: Return an object with properties for
  //       monthlyPayment, totalPayments, totalPaid
}
```

2 Start by converting the percentage rate into a decimal rate, then into a monthly rate.

~/homework/05/practice.js

```
function calculateLoan(principal, annualRate, years) {
-  // TODO: Return an object with properties for
-  //       monthlyPayment, totalPayments, totalPaid
+  const rate = annualRate / 100;
+  const monthlyRate = rate / 12;
}
```

For example, an `annualRate` of `4.45` means `4.45%`. Dividing by `100` gives us `0.0445`, and dividing by `12` gives us the monthly rate.

3 Calculate the number of payments.

~/homework/05/practice.js

```
function calculateLoan(principal, annualRate, years) {
  const rate = annualRate / 100;
  const monthlyRate = rate / 12;
+  const numberOfPayments = years * 12;
}
```

4 Add the loan formula factor.

~/homework/05/practice.js

```
function calculateLoan(principal, annualRate, years) {
  const rate = annualRate / 100;
  const monthlyRate = rate / 12;
  const numberOfPayments = years * 12;
+
+  const factor = Math.pow(1 + monthlyRate, numberOfPayments);
}
```

`Math.pow()` raises one number to the power of another number. This part of the formula accounts for interest building across all monthly payments.

5 Calculate the monthly payment.

~/homework/05/practice.js

```
function calculateLoan(principal, annualRate, years) {
  const rate = annualRate / 100;
  const monthlyRate = rate / 12;
  const numberOfPayments = years * 12;

  const factor = Math.pow(1 + monthlyRate, numberOfPayments);
+
+  const monthlyPayment =
+    principal * (monthlyRate * factor) / (factor - 1);
}
```

6 Round the monthly payment to two decimal places.

~/homework/05/practice.js

```
function calculateLoan(principal, annualRate, years) {
  const rate = annualRate / 100;
  const monthlyRate = rate / 12;
  const numberOfPayments = years * 12;

  const factor = Math.pow(1 + monthlyRate, numberOfPayments);

  const monthlyPayment =
    principal * (monthlyRate * factor) / (factor - 1);
+
+  const roundedMonthlyPayment = parseFloat(monthlyPayment.toFixed(2));
}
```

The `.toFixed(2)` method formats the number to two decimal places, but it gives back a string. `parseFloat()` converts that result back into a number.

7 Return the final loan summary object.

~/homework/05/practice.js

```
function calculateLoan(principal, annualRate, years) {
  const rate = annualRate / 100;
  const monthlyRate = rate / 12;
  const numberOfPayments = years * 12;

  const factor = Math.pow(1 + monthlyRate, numberOfPayments);

  const monthlyPayment =
    principal * (monthlyRate * factor) / (factor - 1);

  const roundedMonthlyPayment = parseFloat(monthlyPayment.toFixed(2));
+
+  return {
+    monthlyPayment: roundedMonthlyPayment,
+    totalPayments: numberOfPayments,
+    totalPaid: parseFloat((roundedMonthlyPayment * numberOfPayments).toFixed(2))
+  };
}
```

8 Add a temporary test call below `calculateLoan()`.

~/homework/05/practice.js

```
function calculateLoan(principal, annualRate, years) {
  const rate = annualRate / 100;
  const monthlyRate = rate / 12;
  const numberOfPayments = years * 12;

  const factor = Math.pow(1 + monthlyRate, numberOfPayments);

  const monthlyPayment =
    principal * (monthlyRate * factor) / (factor - 1);

  const roundedMonthlyPayment = parseFloat(monthlyPayment.toFixed(2));

  return {
    monthlyPayment: roundedMonthlyPayment,
    totalPayments: numberOfPayments,
    totalPaid: parseFloat((roundedMonthlyPayment * numberOfPayments).toFixed(2))
  };
}
+
+console.log(calculateLoan(64000, 4.45, 5));

function fromSeconds(totalSeconds) {
```

You should see this result:

Output

```
{ monthlyPayment: 1191.7, totalPayments: 60, totalPaid: 71502 }
```

JavaScript may display `1191.7` instead of `1191.70` because numbers do not remember trailing zeroes. That is fine. The numeric value is still correct.

Checkpoint

`calculateLoan(64000, 4.45, 5)` should return an object with `monthlyPayment: 1191.7`, `totalPayments: 60`, and `totalPaid: 71502`.

Converting Seconds into Time Parts

Our final function, `fromSeconds()`, receives a total number of seconds and returns an object with separate `hours`, `minutes`, and `seconds` properties.

For example, `3695` seconds is:

1. 1 full hour
2. 1 full minute
3. 35 seconds left over

1 Find the `fromSeconds()` function.

~/homework/05/practice.js

```
function fromSeconds(totalSeconds) {
  // TODO: Return an object with properties for
  //       hours, minutes, seconds
}
```

2 Calculate the full number of hours.

~/homework/05/practice.js

```
function fromSeconds(totalSeconds) {
-  // TODO: Return an object with properties for
-  //       hours, minutes, seconds
+  let hours = Math.floor(totalSeconds / 3600);
}
```

We use `Math.floor()` because a partial hour does not count as a full hour.

3 Calculate the full number of minutes left after removing the hours.

~/homework/05/practice.js

```
function fromSeconds(totalSeconds) {
  let hours = Math.floor(totalSeconds / 3600);
+  let minutes = Math.floor(totalSeconds / 60) - hours * 60;
}
```

4 Calculate the remaining seconds.

~/homework/05/practice.js

```
function fromSeconds(totalSeconds) {
  let hours = Math.floor(totalSeconds / 3600);
  let minutes = Math.floor(totalSeconds / 60) - hours * 60;
+ let seconds = totalSeconds - hours * 3600 - minutes * 60;
}
```

5 Return the result object.

~/homework/05/practice.js

```
function fromSeconds(totalSeconds) {
  let hours = Math.floor(totalSeconds / 3600);
  let minutes = Math.floor(totalSeconds / 60) - hours * 60;
  let seconds = totalSeconds - hours * 3600 - minutes * 60;
+ return {
+   hours,
+   minutes,
+   seconds
+ };
}
```

This object uses property shorthand. Writing `hours` inside the object is the same as writing `hours: hours`.

6 Add one more temporary test call.

~/homework/05/practice.js

```
function fromSeconds(totalSeconds) {
  let hours = Math.floor(totalSeconds / 3600);
  let minutes = Math.floor(totalSeconds / 60) - hours * 60;
  let seconds = totalSeconds - hours * 3600 - minutes * 60;
  return {
    hours,
    minutes,
    seconds
  };
}
+
+console.log(fromSeconds(3695));

// DO NOT MODIFY CONTENT BELOW THIS LINE
```

You should see:

```
Output
{ hours: 1, minutes: 1, seconds: 35 }
```

> **Checkpoint**
>
> `fromSeconds(3695)` should return `{ hours: 1, minutes: 1, seconds: 35 }`. You can also try `fromSeconds(60)`, which should return `{ hours: 0, minutes: 1, seconds: 0 }`.

Clean Up the Practice File

The temporary `console.log()` calls helped us inspect the return values while we were building the functions. The automated tests call the functions directly, so those extra log lines are no longer needed.

1. Remove the temporary `console.log()` calls we added during the tutorial.

~/homework/05/practice.js

```
- console.log(soh(3, 5));
- console.log(cah(4, 5));
- console.log(toa(3, 4));
```

~/homework/05/practice.js

```
- console.log(calculateLoan(64000, 4.45, 5));
```

~/homework/05/practice.js

```
- console.log(fromSeconds(3695));
```

2. Confirm that the export statement at the bottom of the file is still present.

~/homework/05/practice.js

```
export { soh, cah, toa, calculateLoan, fromSeconds }
```

3. Add a few comments in your own words above the main sections of your code. You do not need to comment every line, but your comments should show that you understand the purpose of the angle functions, helper functions, loan calculation, and time conversion.

> **Checkpoint**
>
> After cleanup, your script should still run without errors. It may only print the original `Practical Functions - Homework` heading, because the functions are now waiting to be called by the tests.

Run the Tests

If your homework folder includes the test setup, you can run the automated tests from the `tests` folder.

1. Open a terminal in `~/homework/05/tests/`.
2. Run the tests.

   Run from ~/homework/05/tests/

   ```
   pnpm test
   ```

3. Read any failed test carefully. The test names describe which function returned an unexpected value.

> **Testing Tip**
>
> If a test fails, compare the property names first. An object with `degree` instead of `degrees`, or `totalPayment` instead of `totalPayments`, will not match what the tests expect.

Conclusion

You've now practiced writing practical functions that calculate and return useful values. The important idea is not just the trigonometry, the loan formula, or the time conversion. The important programming pattern is that each function receives input, does one focused calculation, and returns a result that other code can use.

You also practiced composing functions by calling `toDegrees()` and `toGradians()` from inside the angle functions. That helper-function pattern is one of the ways we keep programs easier to read and easier to maintain.

Completion Checklist

Before you consider this bucket assignment finished, confirm that:

- `soh()`, `cah()`, and `toa()` return objects with `radians`, `degrees`, and `gradians`.
- `soh()`, `cah()`, and `toa()` use the `toDegrees()` and `toGradians()` helper functions.

- `calculateLoan()` returns an object with `monthlyPayment`, `totalPayments`, and `totalPaid`.
- `fromSeconds()` returns an object with `hours`, `minutes`, and `seconds`.
- The original export statement is still at the bottom of `practice.js`.
- Your temporary `console.log()` test calls have been removed.
- You added useful comments in your own words.
- You recorded roughly how long the assignment took in `Notes.md`.
- You committed your finished work.

Understanding JavaScript

Functions become more useful when they return values instead of only printing output. A returned value can be tested, stored, passed into another function, or combined with other values later in a program.